



FROM *LAPAR* TO *KENYANG*

A Markov Journey Through Open House Delicacies (Hari Raya Special)







MAIN OBJECTIVE

“Use a Markov Decision Process to decide how much to eat at each open house to maximize overall enjoyment while avoiding feeling uncomfortably full”

State	Label	Description
VH	Very Hungry	Ready to devour the buffet
H	Hungry	Stomach is rumbling
S	Satisfied	Comfortable, content
F	Full	Might force a kuih, but that's it
O	Overstuffed	Help me. I shouldn't have eaten that.

Markov Chain

📌 Transition Rules (Simplified)

Skip = No change in state.

Small = Small upward movement in fullness.

Medium = 1–2 step jump.

Large = High chance of going to Full or Overstuffed.

📌 Assumptions

Skip action causes no change (instant house-to-house transition).

Drinks like air sirap are **not included**.

Transition probabilities are **plausible estimates**, not based on real data.

MDP Setup

– Actions and Rewards

Actions: Skip [Eat nothing], Small portion, Medium portion, Large Portion

Rewards: Food enjoyment score (positive), Overstuffed penalty (negative).

Objective: Maximize total enjoyment score over 5 house visits.

R-Programming Implementation

- Using “markovchain” package to utilize Markov model
- Implementing all important components of the MDP
- Simulating a 5 open-house scenario
- Finding optimal policy to optimize open-house feasting


```
### Hari Raya MDP Simulation: From Lapar to Kenyang

# Load libraries
library(markovchain)
library(ggplot2)
library(dplyr)

# Define States and Actions
states <- c("VH", "H", "S", "F", "O")
actions <- c("Skip", "Small", "Medium", "Large")

# Transition Matrices for Each Action
skipMatrix <- matrix(c(
  1, 0, 0, 0, 0,
  0.6, 0.4, 0, 0, 0,
  0.1, 0.2, 0.7, 0, 0,
  0.1, 0.2, 0.3, 0.4, 0,
  0, 0.1, 0.4, 0.5, 0
), 5, byrow = TRUE, dimnames = list(states, states))

smallMatrix <- matrix(c(
  0, 0.8, 0.2, 0, 0,
  0, 0.2, 0.6, 0.2, 0,
  0, 0.1, 0.6, 0.3, 0,
  0, 0, 0, 0.7, 0.3,
  0, 0, 0, 0.1, 0.9
), 5, byrow = TRUE, dimnames = list(states, states))
```



```

print(skipMatrix)

Tmat <- list(Skip = skipMatrix,
            Small = smallMatrix,
            Medium = mediumMatrix,
            Large = largeMatrix)

# Reward Function
baseEnjoy <- c(Skip = 0, Small = 1, Medium = 2, Large = 3)
penalty <- -3
rewardFun <- function(action, nextState) {
  baseEnjoy[action] + ifelse(nextState == "0", penalty, 0)
}

# One-Step Simulation
stepOnce <- function(currState, action) {
  probs <- Tmat[[action]][currState, ]
  nextState <- sample(states, 1, prob = probs)
  reward <- rewardFun(action, nextState)
  list(nextState = nextState, reward = reward)
}

# Policies
policyAlwaysLarge <- function(state, t) "Large"
policyListenTummy <- function(state, t) {
  switch(state, VH = "Large", H = "Medium",
         S = "Small", F = "Skip", 0 = "Skip")
}
policyAltLargeSkip <- function(state, t) {
  if (t %% 2 == 1) return("Large") else return("Skip")
}

```



```
# Compute Optimal Policy
H <- 5
V <- matrix(0, nrow = 5, ncol = H + 1, dimnames = list(states, 0:H))
Pi <- matrix("", nrow = 5, ncol = H, dimnames = list(states, 1:H))
for (t in H:1) {
  for (s in states) {
    Q <- sapply(actions, function(a) {
      probs <- Tmat[[a]][s, ]
      expR <- sum(probs * sapply(states, rewardFun, action = a))
      expR + sum(probs * V[, t + 1])
    })
    best <- which.max(Q)
    Pi[s, t] <- actions[best]; V[s, t] <- Q[best]
  }
}
policyOptimal <- function(state, t) {
  Pi[state, as.character(t)]
}
```



```
# Simulation Runner and Plotter
run_and_plot_policy <- function(policyFun, policyName) {
  set.seed(42)
  s <- "VH"
  df <- data.frame(House = integer(0),
                   State = character(0),
                   Action = character(0),
                   Reward = numeric(0))

  for (t in 1:5) {
    a <- as.character(policyFun(s, t)) # Ensure action is character
    step <- stepOnce(s, a)
    df <- rbind(df, data.frame(House = t,
                              State = step$nextState,
                              Action = a,
                              Reward = step$reward))

    s <- step$nextState
  }

  print(df)
```



```

p1 <- ggplot(df, aes(x = factor(House), y = State, group = 1)) +
  geom_line(linewidth = 1.2, color = "black") +
  geom_point(size = 3) +
  labs(title = paste("Fullness Journey (", policyName, ")"),
       x = "House", y = "Fullness State") +
  theme_minimal(base_size = 14)

p2 <- ggplot(df, aes(House, cumsum(Reward))) +
  geom_line(color = "darkblue", linewidth = 1.2) +
  geom_point(size = 3, color = "darkblue") +
  labs(title = paste("Cumulative Reward (", policyName, ")"),
       x = "House", y = "Total Enjoyment") +
  theme_minimal(base_size = 14)

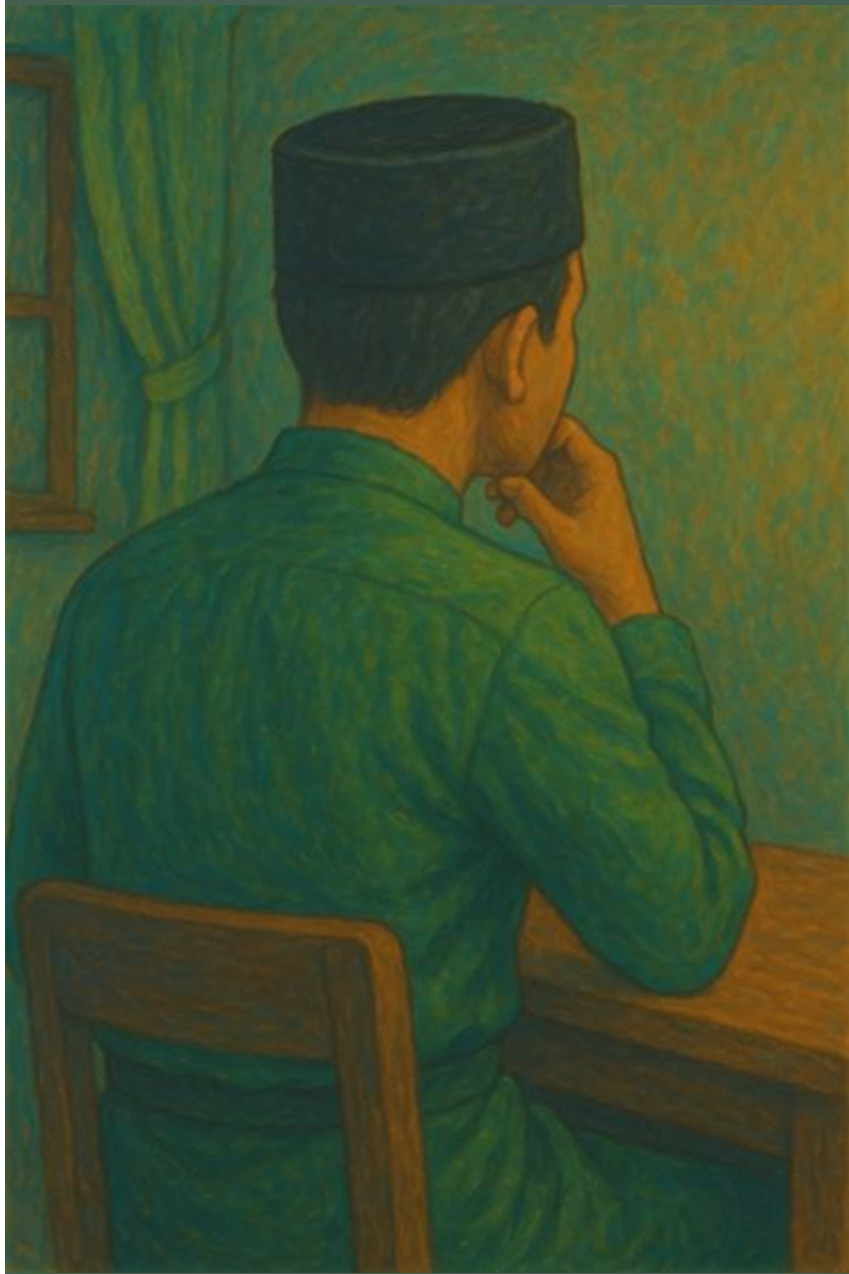
# Run and Plot for Tummy Whisperer and Alt Large-Skip
reward_tummy <- run_and_plot_policy(policyListenTummy, "Tummy Whisperer")
reward_altskip <- run_and_plot_policy(policyAltLargeSkip, "Alt Large-Skip")

cat("\nTotal Rewards:")
cat("\n  Tummy Whisperer:", reward_tummy)
cat("\n  Alt Large-Skip:", reward_altskip)

# Run and Plot for Optimal Policy
reward_optimal <- run_and_plot_policy(policyOptimal, "Optimal")

cat("\nTotal Rewards:")
cat("\n  Optimal Policy:", reward_optimal)
cat("\n  Tummy Whisperer:", reward_tummy)
cat("\n  Alt Large-Skip:", reward_altskip)

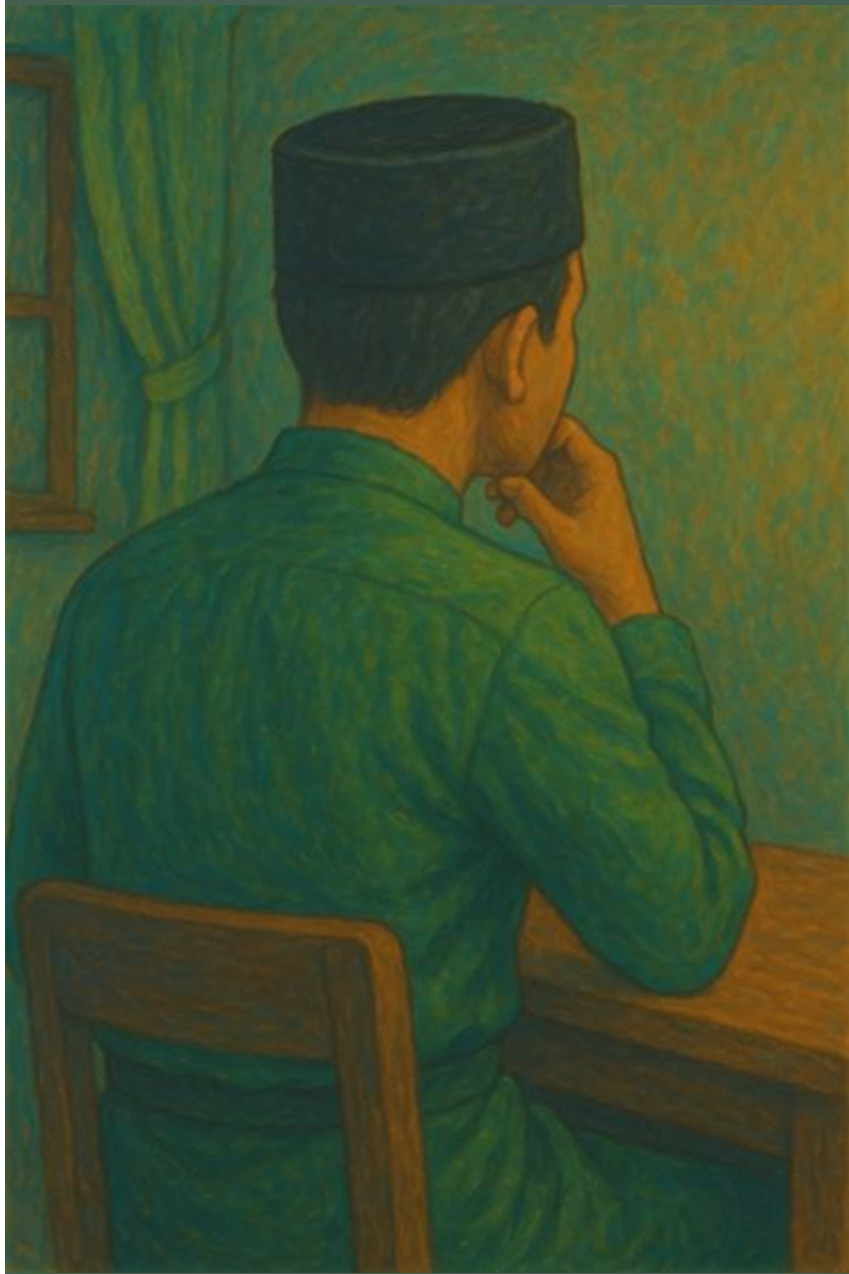
```

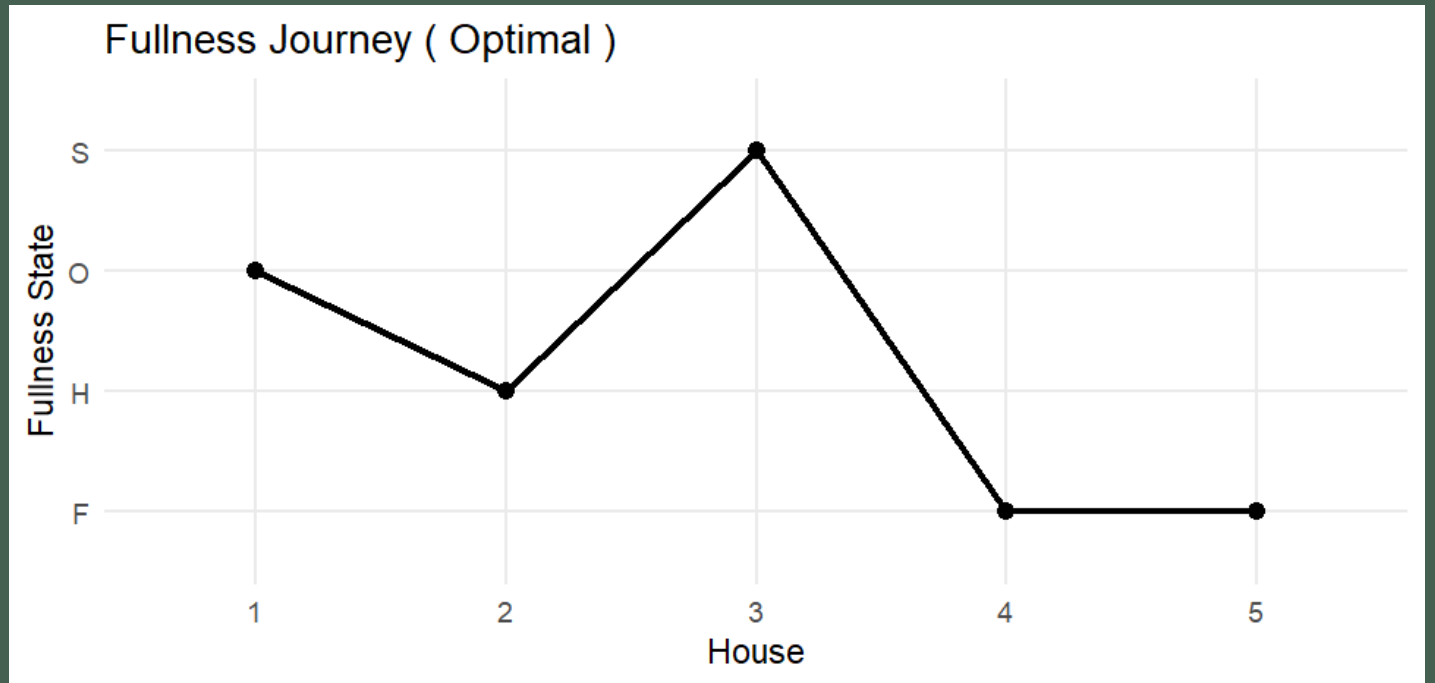
Scenario and Plots

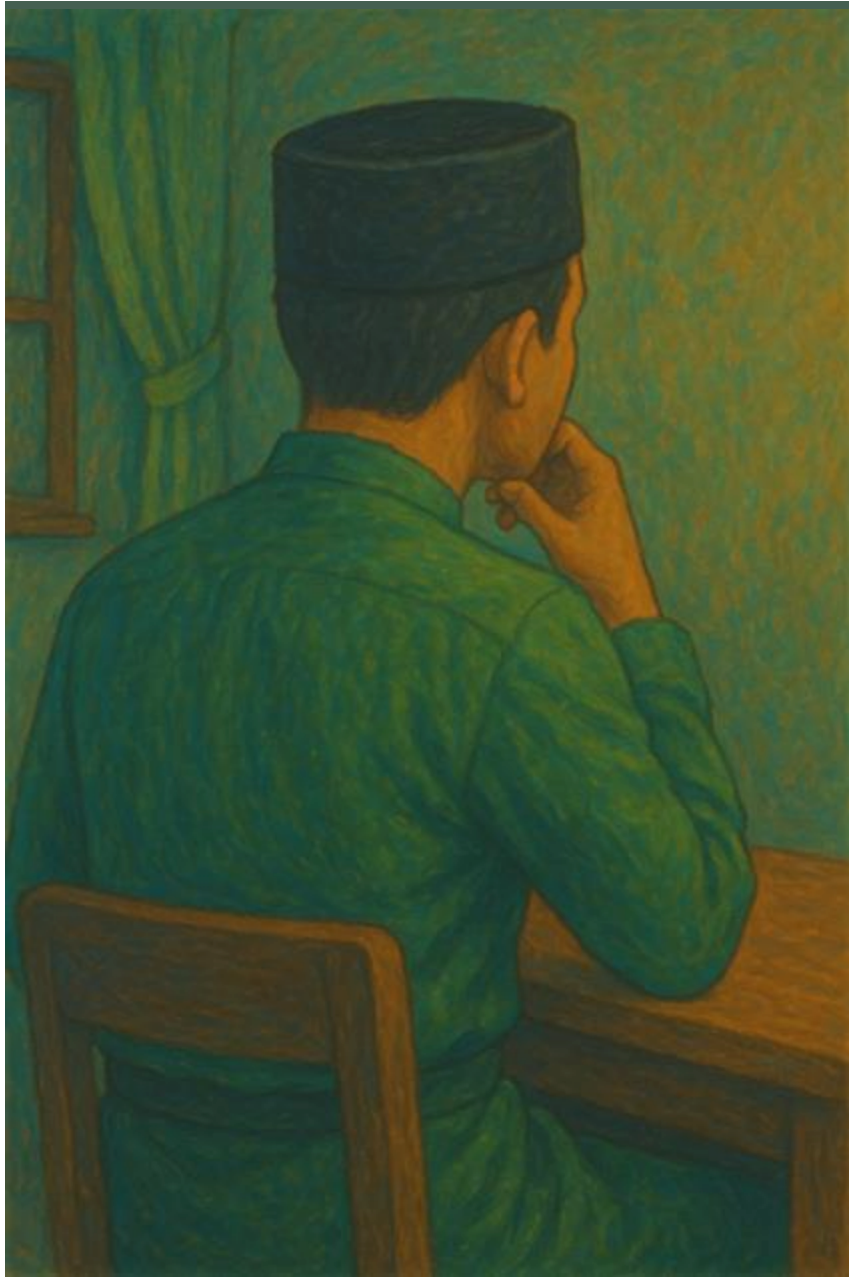
“ If I know I will be visiting 5 open-houses for today, which eating strategy would be the best ?”

- a) The calculated optimal strategy*
- b) Alternating with eating Large portions and Skipping*
- c) Follow my tummy*



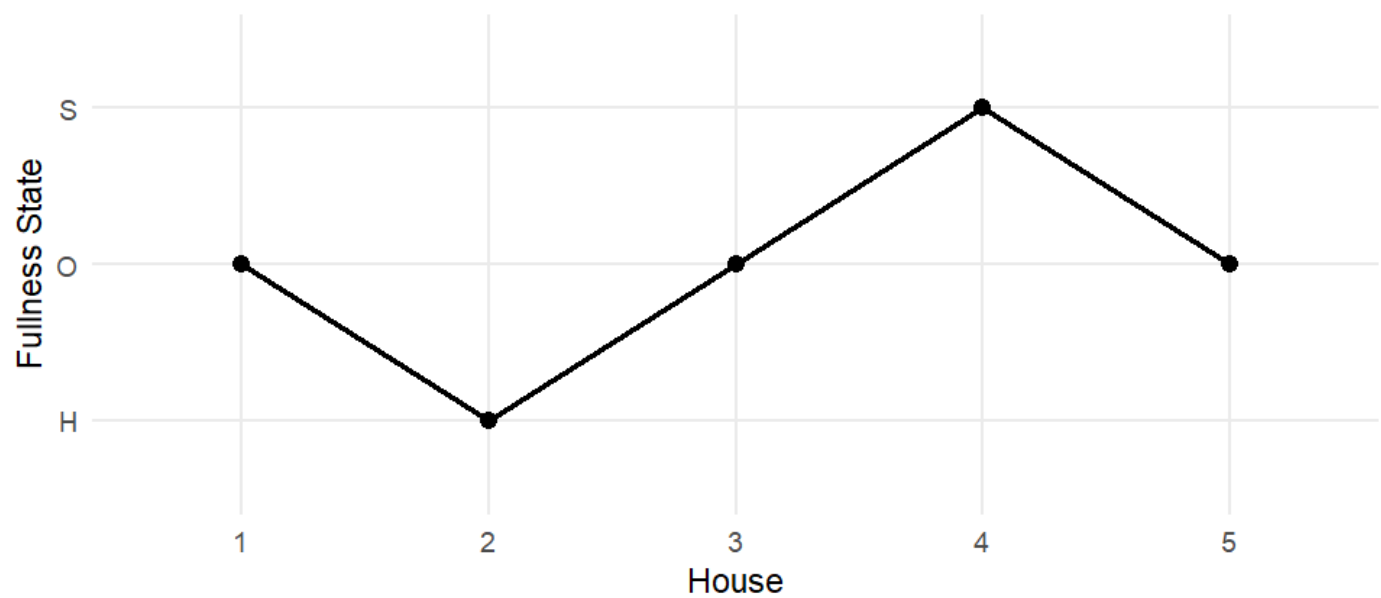
Scenario and Plots

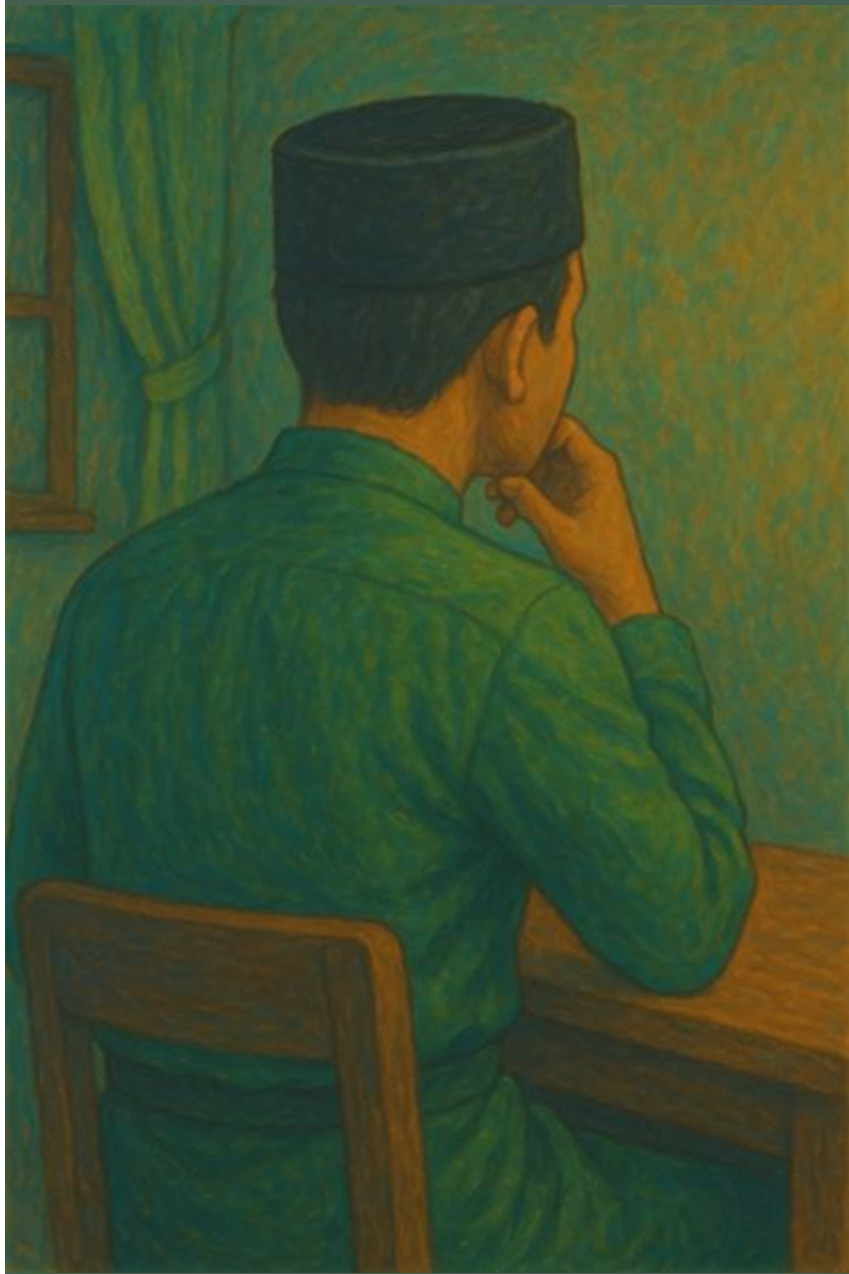




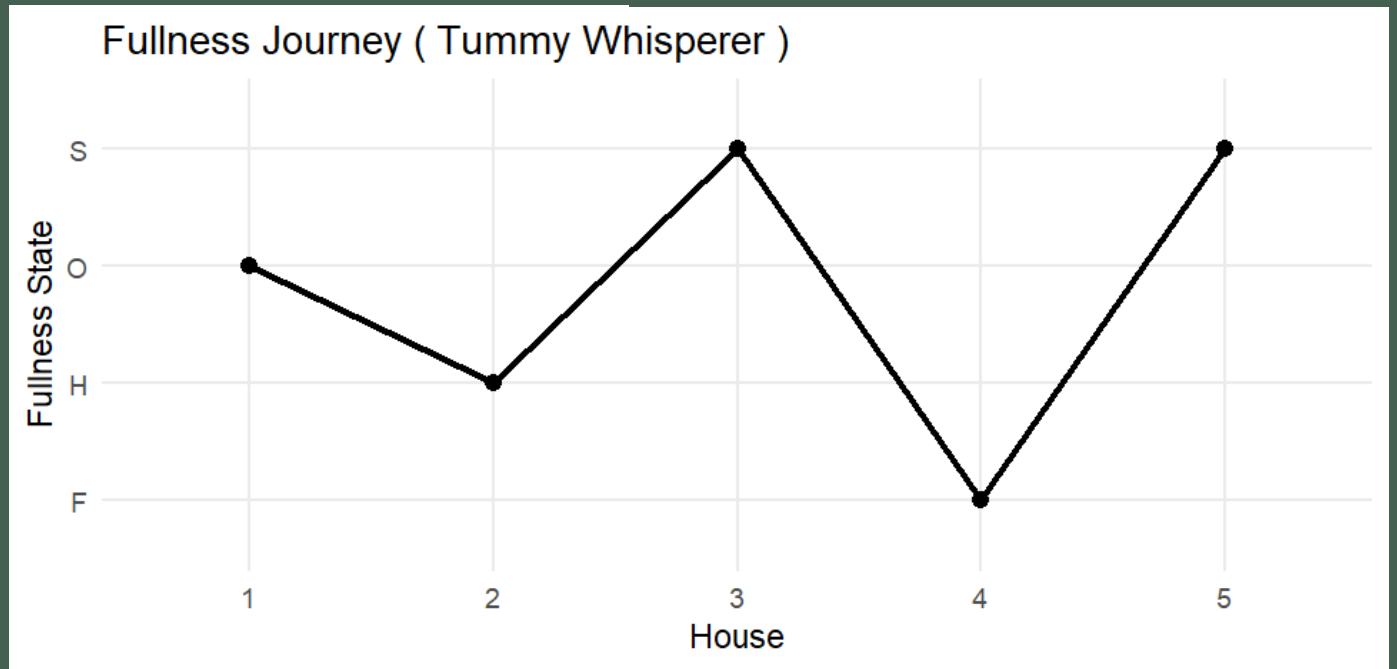
Scenario and Plots

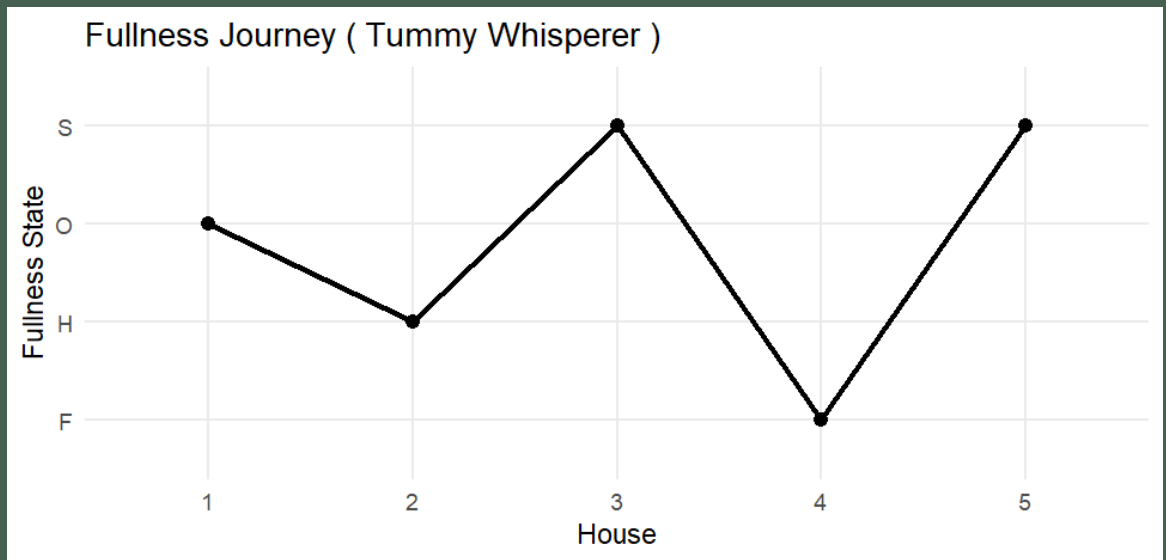
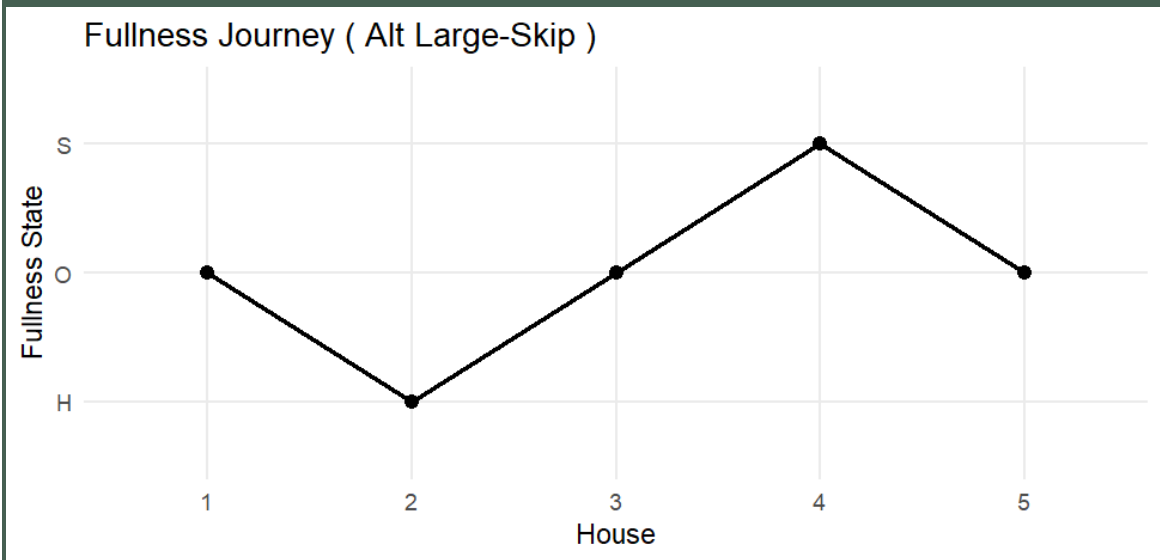
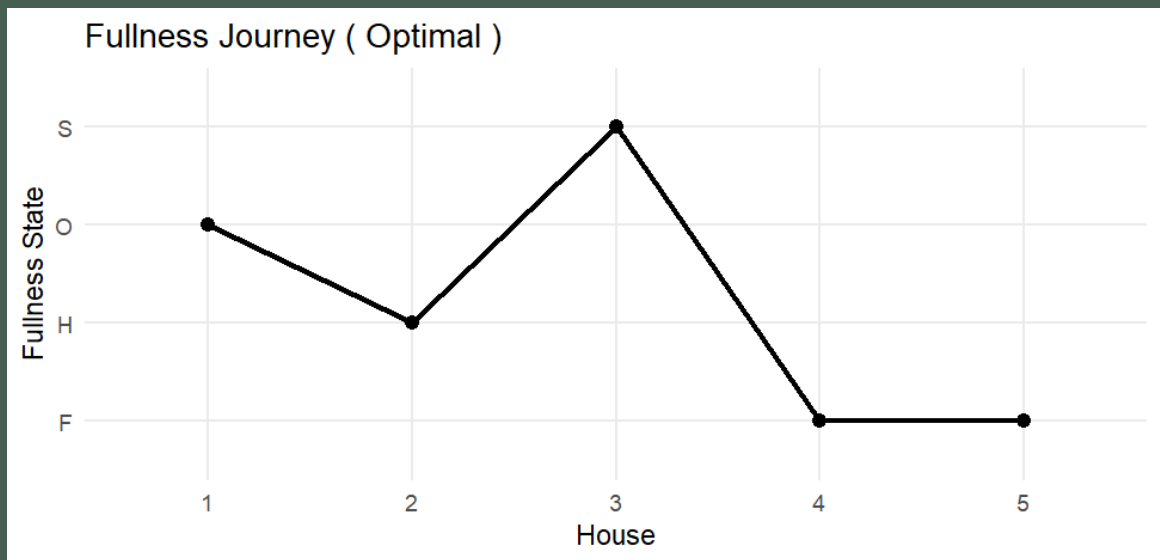
Fullness Journey (Alt Large-Skip)

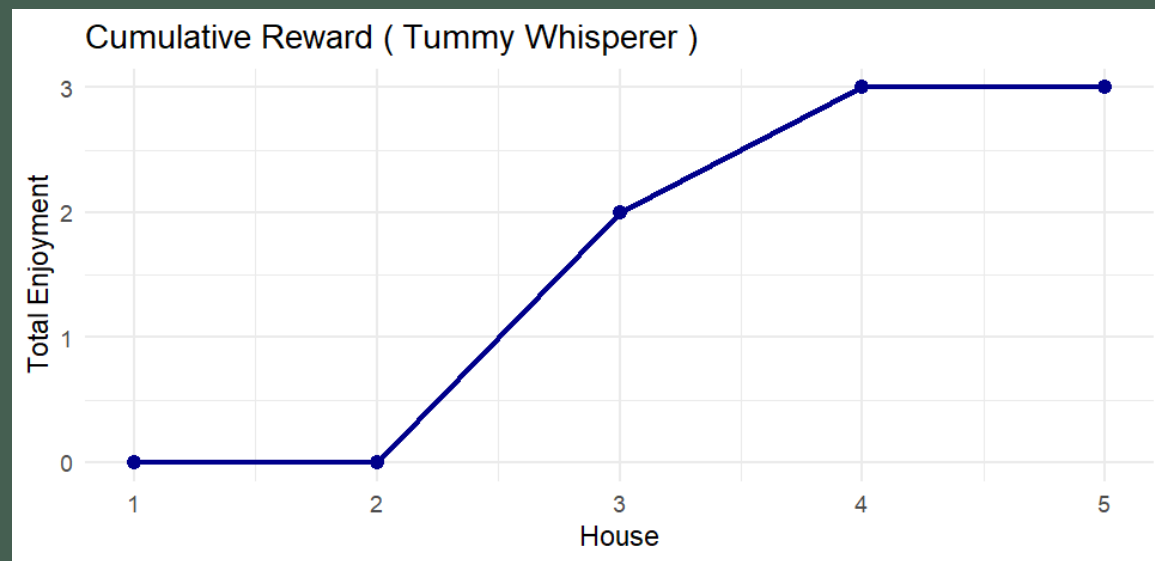
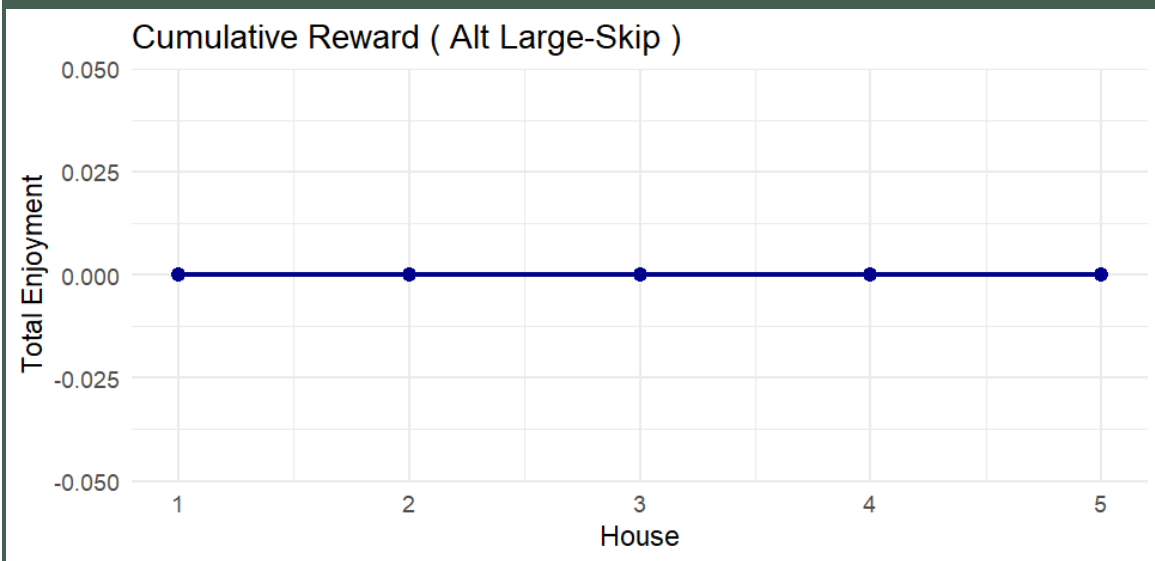
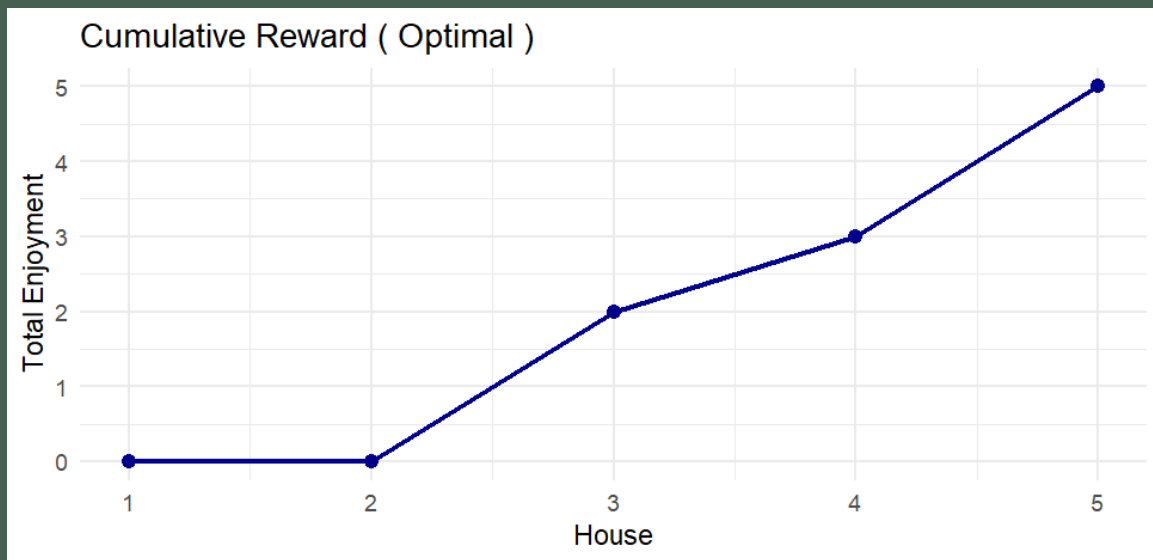


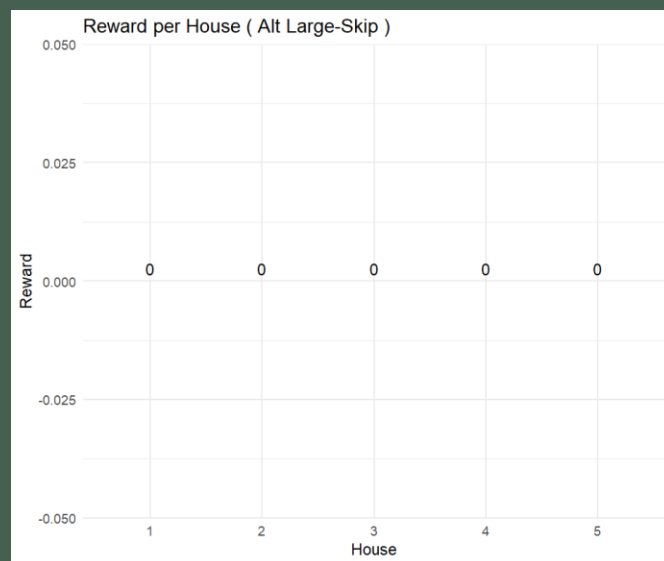


Scenario and Plots

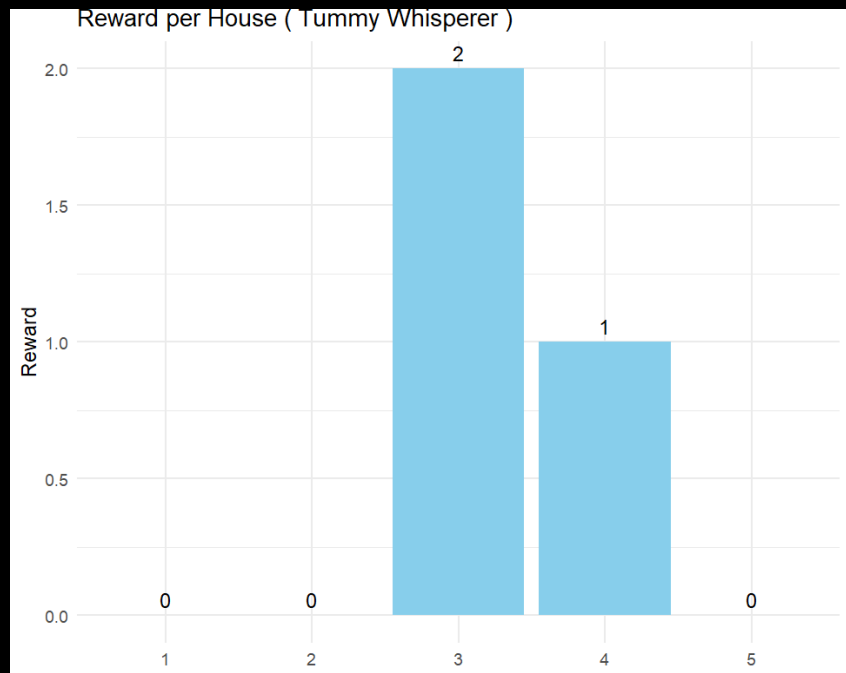
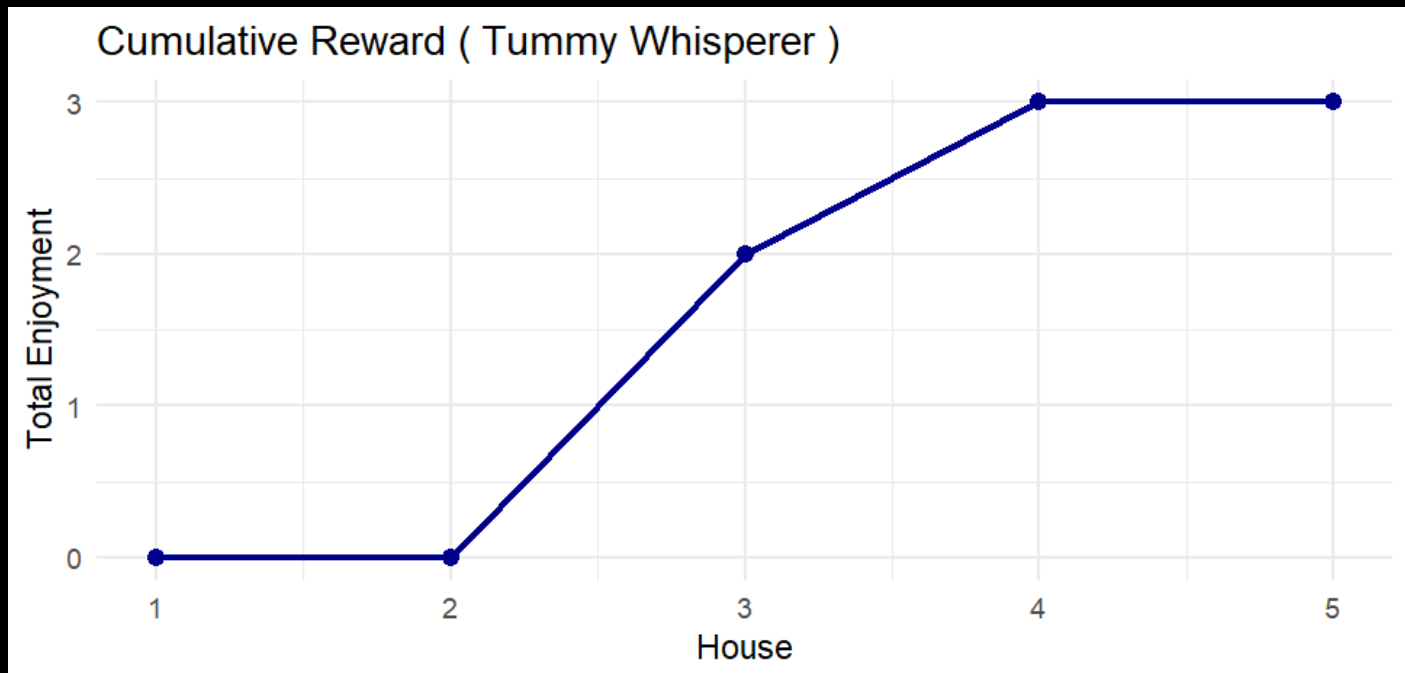
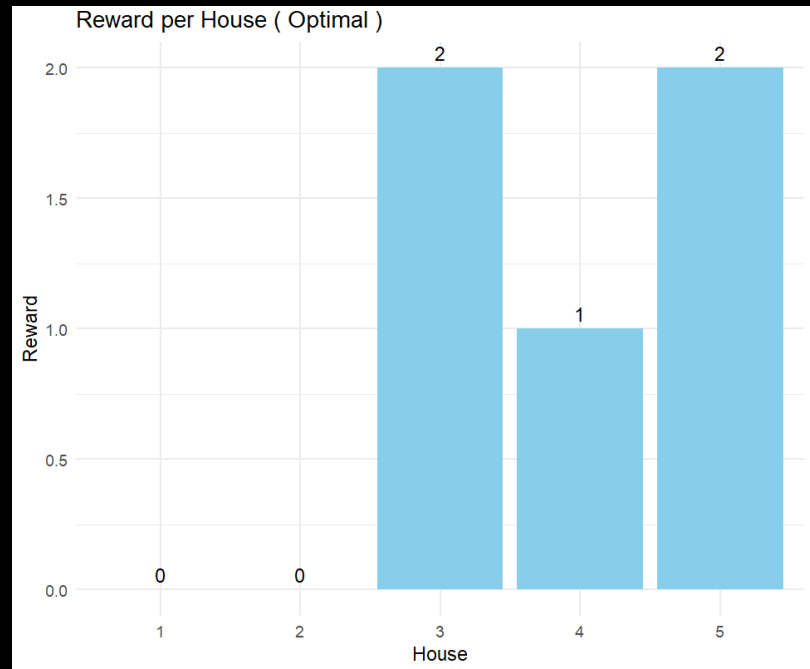
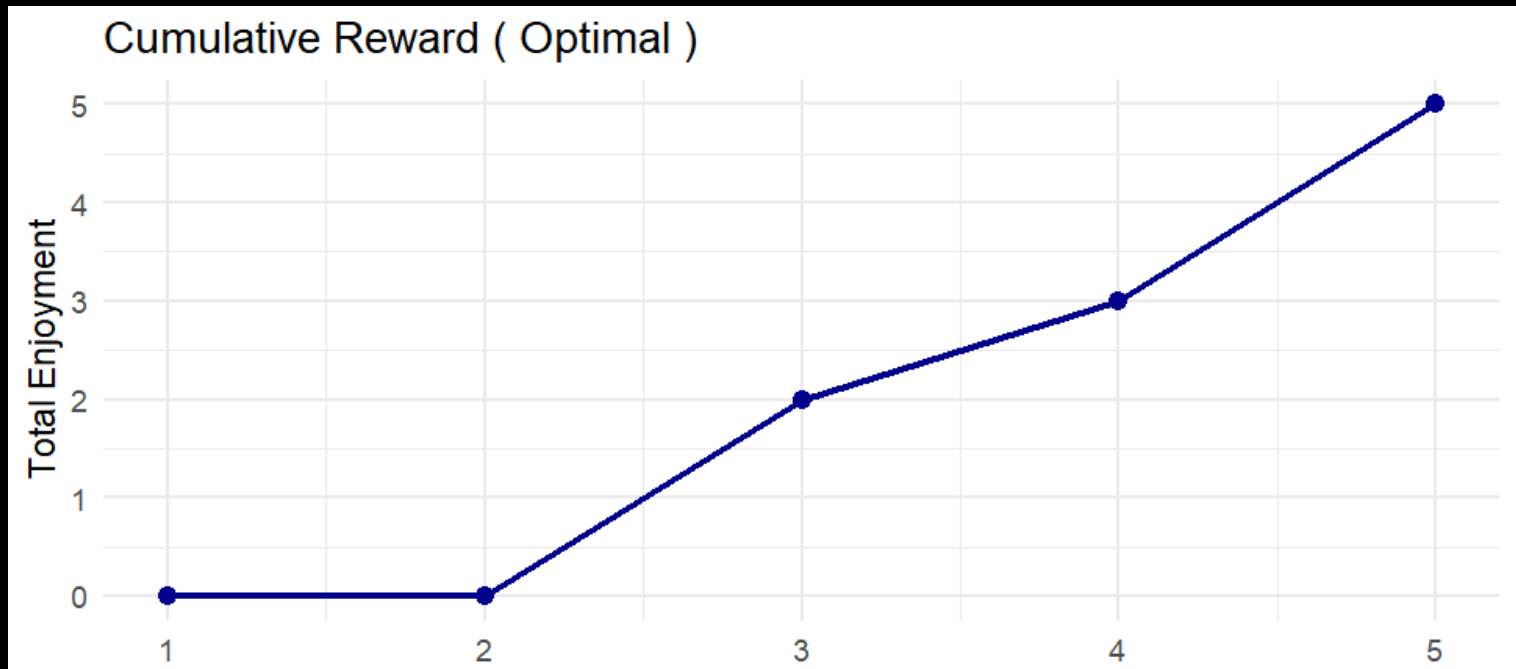






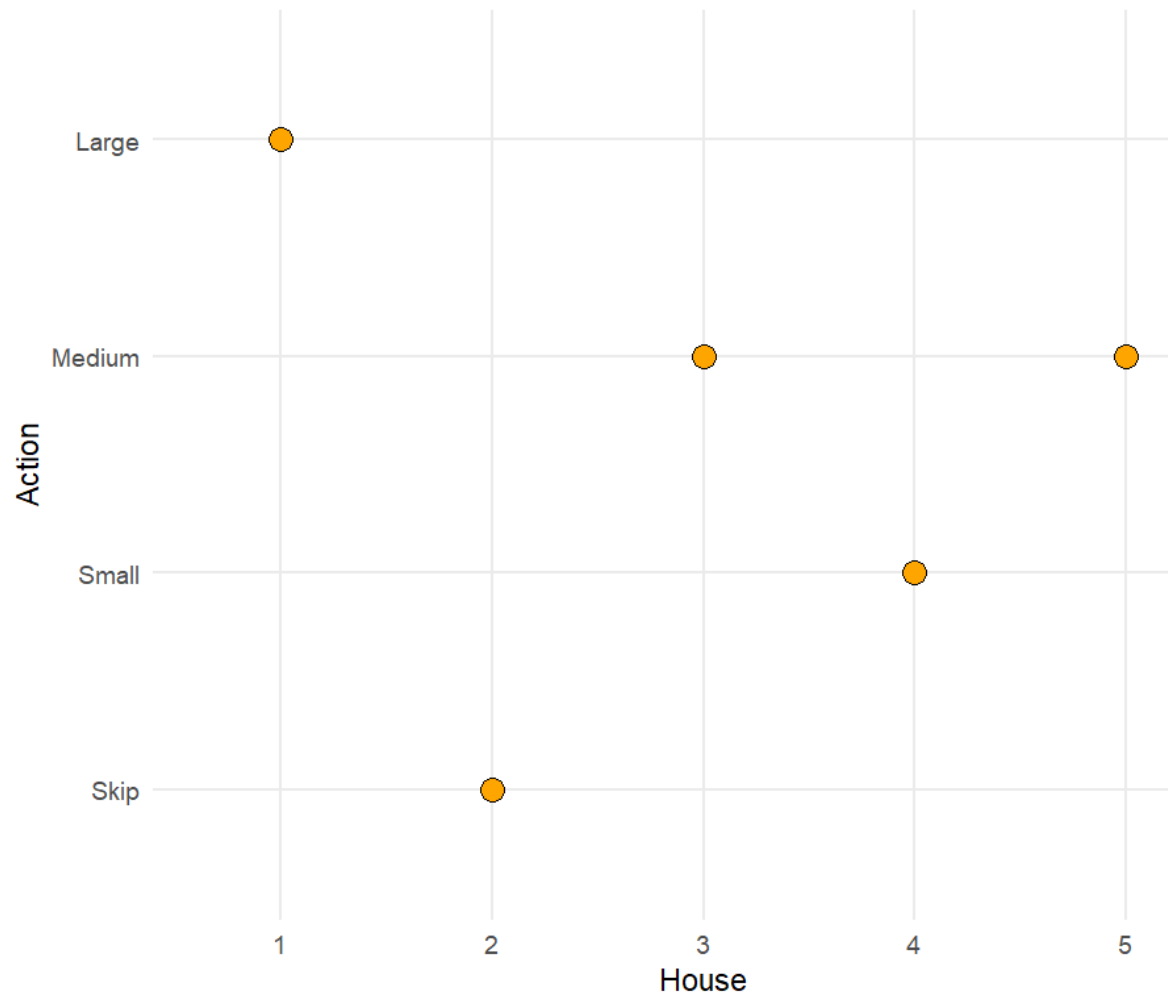




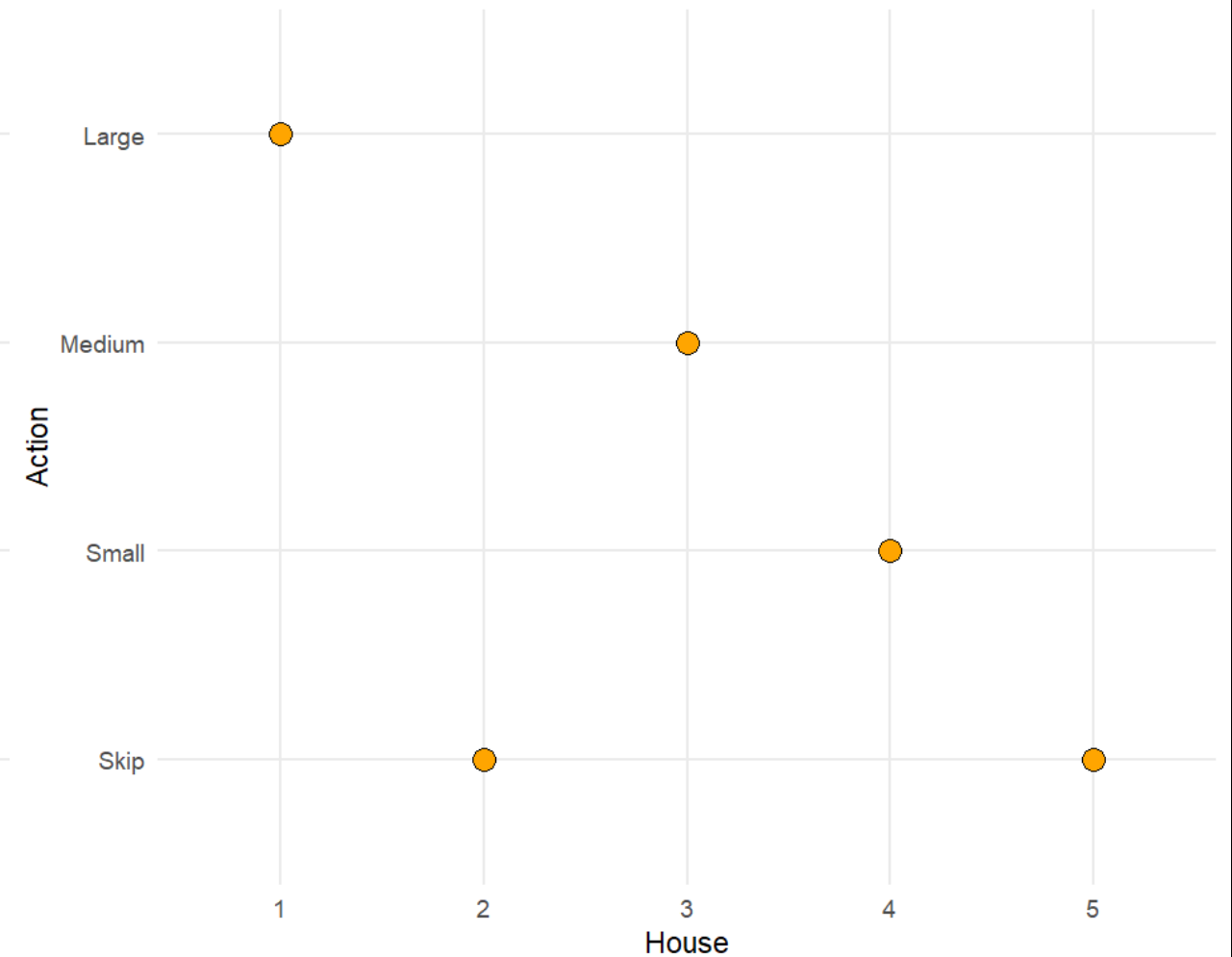


Comparing Strategies

Actions Taken (Optimal)



Actions Taken (Tummy Whisperer)



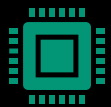
Key Takeaways



Markov Chains + MDP provide structured decision-making for fun scenarios.



Applicable to real-world planning, logistics, health models and predictive analysis



Be ready to explain probabilities, assumptions, and scalability.

An illustration of a family of four—father, mother, and two children—waving their hands. They are dressed in traditional Malay attire: the father in a blue long-sleeved shirt and a black songkok, the mother in a blue long-sleeved shirt and a blue hijab, and the children in matching blue shirts. They are standing in front of a wooden door with a yellow rectangular panel at the top. The scene is lit by two lit candles in holders on either side. The entire illustration is framed by a decorative border of green leaves.

*THANK YOU AND SELAMAT
AIDILFITRI*